

Evaluating Machine Learning-Based Intrusion Detection Models Under Distribution Shift

Author: Dhansika.R **Affiliation:** Independent Researcher **Date:** September 2026 **Correspondence:** dhansika.r.cs@gmail.com **Preprint note:** This manuscript has not been published or submitted elsewhere; it is posted as a preprint and formatted for peer-review submission.

Abstract

Machine learning-based network intrusion detection systems (NIDS) are widely deployed in cloud environments. The standard evaluation methodology trains a model on a labeled dataset, splits it randomly, and reports performance metrics assuming training and deployment data follow the same distribution — an assumption that frequently fails in real networks. We study four machine learning models (Logistic Regression, Random Forest, XGBoost, Multi-Layer Perceptron) under distribution shifts that reflect realistic deployment, compared with conventional random-split evaluation. Using the CICIDS2017 dataset, we design four evaluation scenarios — conventional random split, a joint session and attack-family distribution shift, attack-family leave-one-out, and synthetic feature perturbation — along with a matched-size control that isolates distribution shift from training set size, and a single-class covariate-shift control that separates seen-class feature displacement from unseen-class generalization. Shift magnitude is quantified with per-feature Wasserstein distances; a threshold sweep tests whether recalibrating the decision threshold can recover collapsed F1. Our key finding is that models achieving near-perfect F1-scores (0.999+) under random splitting can suffer severe performance degradation under distribution shift — with Random Forest dropping from $F1 = 0.9997$ to $F1 = 0.0000$ — while a matched-size control using the same number of training samples on same-distribution data yields $F1 = 0.9992$, providing strong evidence that the degradation is distributional rather than a data-scarcity effect. The failure modes differ by architecture: tree-based models collapse to predicting all traffic as benign, while linear and neural models retain partial detection capability. ROC-AUC can remain partially informative (0.819 for Random Forest) while F1 collapses, indicating retained discriminative signal despite broken operational behavior. A threshold sweep confirms that much of the lost F1 can be recovered (Random Forest: 0.0000 to 0.826), but only at far-below-default thresholds that carry a substantial false-alarm burden.

Keywords: intrusion detection, distribution shift, robustness evaluation, machine learning, cybersecurity, concept drift

Highlights - Random-split F1 of 0.9997 drops to 0.0000 under session/attack-family shift. - Matched-size control proves degradation is distributional, not data scarcity. - Covariate-shift control isolates unseen classes as the main cause of collapse. - Tree models keep ranking signal but need far-below-default thresholds to use it. - Threshold recalibration is a partial remedy, not a restoration of performance.

1. Introduction

Cloud computing infrastructure has transformed the scale and complexity of network environments. Modern cloud deployments involve thousands of interconnected services, dynamic virtual machines, containerized workloads, and traffic patterns that shift continuously in response to user behavior, application updates, and emerging threats. This migration introduces challenges for network security monitoring: the attack surface grows with every new service, the volume of network traffic scales with user demand, and the heterogeneity of traffic patterns makes it difficult to distinguish benign from malicious activity using static rules alone.

Network intrusion detection systems (NIDS) sit between a network and its defenders: they watch traffic flows and try to catch malicious activity — brute-force logins, denial-of-service floods, data exfiltration, lateral movement. Signature-based tools only catch what they have been told to look for, so they miss attacks they have never seen. That is why ML-based detection is attractive: it learns complex patterns and is expected to generalize to new attack variants. A small set of models dominates — Logistic Regression, Random Forest,

gradient-boosted trees, neural networks — and reports exceed 0.99 F1 in study after study.

Look closely at how those numbers are produced, though. The standard recipe splits one labeled dataset into random training and test subsets. Because the split is random, both halves come from the same underlying distribution, with similar class proportions and similar feature statistics. Under those conditions near-perfect scores are exactly what should happen. The question is whether they survive deployment, where identical training and test distributions never actually exist.

The project asks one research question:

RQ: To what extent does conventional random-split evaluation overestimate the operational effectiveness of machine learning-based intrusion detection models under distribution shift?

Four sub-questions structure the experiments:

- **RQ1 (Experiment B):** Does distribution shift reflecting realistic deployment conditions cause material performance degradation relative to random-split evaluation?
- **RQ2 (Experiment B2):** Is any such degradation attributable to distributional differences rather than insufficient training data?
- **RQ3 (Experiment B4):** Is the degradation driven primarily by unseen attack classes or by covariate shift?
- **RQ4 (Section 7.8):** To what degree can decision-threshold recalibration recover collapsed performance, and at what false-alarm cost?

The study covers three shift types relevant to network security: **session and attack-family shift** (training and test data from sessions with different attack compositions), **attack-family shift** (an attack type entirely absent from training), and **synthetic feature perturbation** (approximating infrastructure-level changes in observed features). These represent different dimensions of the shift problem: temporal/session variation, attack-space coverage, and feature-level changes.

If evaluation methodology does not account for distribution shift, benchmark numbers will keep overstating how well these models actually perform in the field.

The paper makes four contributions. First, a controlled experimental framework that isolates distribution shift from confounding variables such as training set size, using matched-size controls with zero train/test overlap and a single-class covariate-shift control separating unseen attack classes from feature displacement. Second, a quantitative characterization of the shifts using per-feature Wasserstein distances, plus evidence that near-perfect benchmark performance can mask severe vulnerability to distribution shift, with tree-based models dropping from $F1 = 0.9997$ to $F1 = 0.0000$. Third, a demonstration that the four evaluated architectures fail differently — tree-based models fail by predicting all traffic as benign, while linear and neural models retain partial detection. And fourth, a threshold-sweep analysis showing that the retained ROC-AUC signal can be converted back into classification performance only at far-below-default decision thresholds, with a measurable false-alarm cost.

2. Problem Statement

The core problem is a gap between how ML-based intrusion detection models are evaluated and how they are deployed. In evaluation, models are trained and tested on data from the same distribution. In deployment, they encounter traffic that differs from training in temporal patterns, attack composition, and network configuration. The resulting performance discrepancy can lead to a dangerous false sense of security: a model that appears highly effective in benchmarks may fail silently in production.

Consider a concrete scenario: an organization deploys a Random Forest classifier trained on network traffic from one week of operation. The model achieves 99.97% F1-score on a held-out test set from the same week. Two months later, the network experiences a DDoS attack that the model has never encountered during training. The model, having learned decision boundaries specific to the training distribution, classifies every flow as benign — providing zero detection capability while the organization believes its ML-based IDS is operational. This scenario is not hypothetical: it is a predictable consequence of evaluating models under conditions that do not match their deployment environment. Conventional metrics (accuracy, F1, ROC-AUC) are reported without any indication of how robust they are to distributional changes. A paper reporting "F1 =

0.9997 on CICIDS2017" provides no information about whether this performance will persist when the traffic distribution shifts — which it inevitably will in any production network. The problem is amplified in cloud environments, where traffic patterns are inherently dynamic and where traffic is aggregated from multiple tenants and services, creating composite distributions that are less predictable than single-organization networks. An ML-based IDS trained on traffic from one configuration must generalize to configurations it has never seen, and the degree to which it can do so is precisely what conventional evaluation fails to measure.

3. Research Gap

The existing body of work on ML-based intrusion detection includes hundreds of studies reporting high performance on benchmark datasets. However, three gaps limit the practical value of these results.

First, distribution shift is rarely evaluated. The dominant practice is a single random train/test split, which does not test robustness to distributional changes. Studies that do evaluate under shift often use different datasets for training and testing (cross-dataset evaluation), where different feature extraction methods, preprocessing pipelines, and network configurations confound the comparison. What is needed are controlled experiments within a single dataset that systematically vary the distribution while holding other factors constant.

Second, confounds are rarely isolated. When a model performs poorly under distribution shift, it is unclear whether the cause is insufficient training data, the distributional difference itself, or their interaction. Without controlled experiments that vary these factors independently — for example, a matched-size control that trains on the same number of samples from the same distribution as the test set — the conclusions are ambiguous.

Third, failure modes are rarely analyzed. The pattern of failure — whether the model becomes conservative (predicting mostly benign), aggressive (predicting mostly attack), or random — has different security implications and requires different remediation strategies, but such analysis is frequently absent from the literature.

This paper addresses all three gaps through a controlled experimental design with matched-size controls, multiple shift types, and detailed failure mode analysis.

4. Related Work

4.1 Machine Learning for Network Intrusion Detection

ML-based intrusion detection has been studied for over two decades. Recent work has applied diverse model architectures to benchmark datasets. Tavallaee et al. (2009) analyzed the KDD Cup 99 dataset and identified inherent biases that motivated the development of NSL-KDD, which addressed these issues and enabled more reliable benchmarking. Sharafaldin et al. (2018) developed the CICIDS2017 dataset, which includes modern attack types and realistic traffic patterns generated using the CICFlowMeter tool. These datasets have become standard benchmarks, with hundreds of studies evaluating classifiers on them.

Common approaches include Random Forest, gradient-boosted methods such as XGBoost, deep learning architectures (MLPs, CNNs, autoencoders), and traditional methods such as Logistic Regression and Support Vector Machines. Performance on these benchmarks frequently exceeds 0.99 F1-score under conventional evaluation protocols. While this performance appears impressive, the evaluation methodology — a single random train/test split — limits the conclusions that can be drawn about real-world deployment, where the traffic distribution evolves over time.

4.2 Distribution Shift in Machine Learning

Distribution shift — the phenomenon where the statistical properties of data change between training and deployment — is a well-studied problem in machine learning. It encompasses covariate shift (change in input distribution while $P(Y|X)$ remains constant), prior probability shift (change in class proportions), and concept shift (change in the relationship between inputs and labels). All three occur in network security: network configurations change (covariate), attack prevalence varies (prior), and attack characteristics evolve as adversaries develop new techniques (concept).

Shyaa et al. (2024) surveyed 71 studies on concept drift in intrusion detection, classifying drift types as sudden, gradual, incremental, and recurrent, and identifying domain-specific causes including protocol mutation, attacker strategy evolution, and infrastructure changes. They noted that drift detection methods from the general ML literature (ADWIN, DDM, Page-Hinkley) vary in effectiveness by drift type and network context, suggesting that a single drift detection strategy may be insufficient for comprehensive IDS robustness.

4.3 Temporal Evaluation of IDS

Intrusion detection is inherently a time-series problem: models trained on past traffic must classify future traffic, whose statistical properties evolve over time. Ahmad et al. (2021) developed an adaptive system that detects concept drift and triggers retraining when performance degrades, but it requires continuous labeled data — a significant operational constraint, since ground-truth labels are often delayed or unavailable in production. Nibouchia et al. (2024) systematically reviewed how temporal evaluation is handled in IDS publications, finding that many studies do not account for temporal ordering, potentially inflating performance estimates. Our work complements these studies with controlled experiments that quantify degradation under session-based shift and isolate its cause through matched-size controls.

4.4 Adversarial Robustness of NIDS

Adversarial robustness concerns deliberate modification of traffic to evade detection. While our study examines natural distribution shift (session differences, attack-family variation) rather than adversarial manipulation, the broader robustness framework is relevant: Coronges et al. (2024) surveyed over 200 papers identifying evasion, poisoning, and model extraction as primary threat vectors, and noted that robustness against natural shifts and adversarial attacks are complementary aspects of reliable detection under conditions that differ from the training environment.

4.5 Concept Drift Adaptation

Drift adaptation methods include sliding windows, ensembles, and online learning. Sadhnani et al. (2024) used Explainable AI to identify when and why model behavior changes, finding that different attack types drift at different rates, which argues against a single global retraining schedule. Akinola et al. (2022) developed a feedback mechanism that updates models in real time based on analyst decisions, addressing the unavailability of ground-truth labels in operations. These approaches address how to detect and respond to shift, whereas we evaluate the degradation that occurs when shift is not handled.

4.6 Summary

This paper contributes a controlled experimental framework that isolates the effects of distribution shift, compares the robustness of multiple model architectures under identical conditions, and connects quantitative degradation patterns to practical security implications. Unlike prior work that either evaluates a single shift type or does not control for training set size, our approach uses matched-size controls with zero train/test overlap to establish attribution for the observed degradation.

5. Dataset and Experimental Setup

5.1 Dataset

We use the Friday subset of the CICIDS2017 dataset (Sharafaldin et al., 2018), a widely used benchmark for intrusion detection research. Preprocessing first replaces infinite values with NaN and drops rows containing NaN/Inf: 527 rows (0.07% of the raw 703,245) were removed, and no imputation was applied. Non-numeric columns are dropped, leaving 78 numeric features. The resulting dataset contains 318,237 rows with 4 classes:

Class	Count	Percentage
BENIGN	29,452	9.3%
Bot	1,956	0.6%
DDoS	128,025	40.2%
PortScan	158,804	49.9%

Features include flow-level statistics (Flow Duration, Total Fwd/Backward Packets, packet size statistics), flag-based features (SYN/ACK/RST counts), and inter-arrival time statistics. All features are numeric and represent aggregated network flow characteristics. The class imbalance is substantial: DDoS and PortScan together account for 90.1% of the data, while Bot traffic represents only 0.6%. This imbalance is realistic — in production networks, specific attack types are typically rare relative to total traffic — and presents a challenge for model training and evaluation that must be accounted for in the experimental design.

5.2 Prediction Task

We primarily evaluate binary classification: BENIGN (0) vs. ATTACK (1). The 4-class problem is evaluated as a supplementary analysis.

5.3 Models

We evaluate four models representing distinct algorithmic paradigms:

- **Logistic Regression (LR):** Linear classifier with L2 regularization; invariant to feature scaling but assumes linear decision boundaries
- **Random Forest (RF):** Ensemble of 80 decision trees (max_depth=15); axis-aligned splits make it invariant to monotonic feature transformations
- **XGBoost:** Gradient-boosted ensemble of 80 trees (max_depth=5, lr=0.1); sequential optimization may increase sensitivity to training distribution
- **MLP:** Two hidden layers (128, 64 units); captures nonlinear feature interactions but requires sufficient training data

All models use fixed hyperparameters with `random_state=42` for reproducibility.

Focus on traditional machine learning. We deliberately confine the study to these four models rather than deep architectures. The motivating deployment context is throughput-constrained cloud IDS: Random Forest, gradient-boosted trees, and logistic regression remain widely deployed in production because they offer near-line-rate inference on commodity CPUs with modest memory and power requirements, whereas deep models typically require GPU acceleration and higher inference latency. The inductive-bias differences matter for the observed failures: Random Forest's axis-aligned splits cannot extrapolate beyond the range of feature values seen in training — consistent with its complete collapse to predicting all traffic as benign when test features fall outside the training envelope — while linear and neural models learn continuous magnitude-sensitive representations that degrade more gracefully. Comparing deep architectures against these models is outside the scope of this study and noted as future work.

6. Methodology

6.1 Evaluation Scenarios

Experiment A: Conventional Random Split. Standard 80/20 stratified split. This provides the conventional benchmark performance metric and serves as a baseline.

Experiment B: Joint Session and Attack-Family Distribution Shift. Training data consists of traffic from the morning session of CICIDS2017 Friday (Bot attacks + randomly sampled benign traffic, 13,736 samples). Test data consists of traffic from the afternoon session (DDoS + PortScan + randomly sampled benign traffic, 298,609 samples): a model trained on one set of sessions encounters different sessions containing different attack types, so the training and test sets differ in both session timing and attack composition. Critically, B is a **joint domain and unseen-class shift**: the test set contains two attack families (DDoS, PortScan) entirely absent from training, conflating covariate shift (session timing, benign composition, feature statistics) with unseen-class generalization (novel attack signatures). Experiment B4 (Section 6.2) isolates the covariate-shift component.

Experiment C: Attack-Family Leave-One-Out. For each attack type (Bot, DDoS, PortScan), the model is trained on all other traffic and tested on the held-out attack type plus a comparable number of unseen benign samples. To prevent data leakage, benign samples used in the test set are explicitly excluded from the training set. This directly evaluates how well models generalize to entirely unseen attack types — a critical capability for real-world deployment where the training set inevitably misses some attack variants.

Experiment D: Synthetic Feature Perturbation. Using the same random split as Experiment A, we apply synthetic perturbations to the test set features: Flow Duration is multiplied by random factors drawn uniformly from 2.0 to 5.0, and packet count features (Total Fwd Packets, Total Backward Packets) are scaled by Gaussian noise (mean 1.5, std 0.5), clipped to $[-10^6, 10^6]$. This tests invariance to feature-level changes arising from infrastructure modifications, sensor recalibration, or changes in flow measurement tools.

6.2 Control Experiments

The critical methodological contribution is the use of matched-size controls to isolate distribution shift from training set size.

Experiment B2: Matched-Size Control. We subsample the random training pool from Experiment A down to 13,736 samples (matching B's training set size) and test on the same held-out random split from Experiment A. This control has: (1) the same training set size as B, (2) the same test set, (3) zero train/test overlap, and (4) training data from the same distribution as the test set. The comparison between B and B2 isolates the effect of distribution shift while controlling for training set size.

Experiment B4: Single-Class Covariate-Shift Control. Experiment B conflates covariate shift with unseen-class generalization; B4 isolates the covariate-shift component. We take only DDoS and BENIGN traffic (157,477 samples), split it 80/20 stratified, and train all models on the training portion — so **DDoS is present in the training set** — testing on the held-out portion. In B4a (no-shift control) the held-out test is used unmodified. In B4b (covariate shift) the same perturbation design as Experiment D is applied to the test features (Flow Duration multiplied by 2–5, packet counts scaled by Gaussian noise centered at 1.5); benign samples are unchanged. Because the attack class is present in training, any degradation between B4a and B4b is attributable to feature displacement (covariate shift) in isolation from unseen-class generalization; the contrast B4b vs. B quantifies how much of B's collapse is due to features outside the training envelope versus entirely novel attack families.

6.3 Evaluation Metrics

We report accuracy, precision, recall, F1-score, FPR, and ROC-AUC for binary classification. For multiclass, we report weighted averages.

Precision: proportion of predicted attacks that are actually attacks.

F1-score: harmonic mean of precision and recall, which provides a single metric that balances false positives and false negatives.

FPR: proportion of benign traffic incorrectly classified as attack. In intrusion detection, this corresponds to the false alarm rate.

6.4 Quantifying Shift Magnitude

To give the distribution shifts a mathematical grounding, we quantify how far the training and test feature distributions have drifted using the one-dimensional Wasserstein distance (Earth Mover's Distance). For each feature, we compute $W1(f) = W(P_{\text{train}}(f), P_{\text{test}}(f))$ on a random subsample of 25,000 points per side and normalize by the pooled standard deviation of the two distributions so that distances are dimensionless and comparable across features. We report the mean normalized Wasserstein distance across all features as a single shift-magnitude index, and the top features driving each shift. We compute this index for the B shift (morning-session training vs. afternoon-session test features), the D shift (unmodified vs. perturbed test features), and the B4 shift (unmodified vs. perturbed DDoS test features), and relate it to the observed F1 drop.

6.5 Threshold Sweep

To test whether the retained ROC-AUC signal can be converted back into usable classification, we sweep the decision threshold on the Experiment B test set from 0.001 to 1.0 in steps of 0.001 for each model and compute F1, precision, recall, and FPR at every threshold. We report the F1 at the default threshold (0.50) versus the best non-degenerate operating point, defined as the maximum F1 over thresholds strictly below 0.50. Because the B test set is 96.1% attack, we also report the FPR and the fraction of samples flagged as attack at each operating point so that the false-alarm cost of aggressive thresholds is explicit.

6.6 Experimental Procedure

All experiments follow the same procedure for each model:

- 1 Fit StandardScaler on training data only; transform test data using the fitted scaler
- 2 Train the model on scaled training data
- 3 Predict labels and probabilities on scaled test data
- 4 Compute all evaluation metrics

Each experiment trains and evaluates all four models independently. All random seeds are fixed (`random_state=42` for `sklearn/xgboost`, `np.random.seed(42)` for `numpy`) to ensure full reproducibility. The shift types were chosen to cover three deployment-relevant dimensions: session/temporal variation, attack-space coverage, and feature-level infrastructure change.

7. Results

7.1 Conventional Random Split (Experiment A)

Under conventional random splitting, all models achieve near-perfect performance (Table 1). Random Forest and XGBoost achieve $F1 = 0.9997$, while Logistic Regression achieves 0.9951 and MLP achieves 0.9989. These results are consistent with the broader literature on CICIDS2017, where models routinely exceed 0.99 F1 under standard evaluation. The precision and recall values are similarly high, with all models achieving precision above 0.998 and recall above 0.996. False positive rates are low across the board (0.003 to 0.061), indicating that the models rarely misclassify benign traffic as attack under these conditions. The inference times differ by an order of magnitude (LR at 0.0001ms vs RF at 0.0014ms per sample); end-to-end deployment latency would also depend on preprocessing, hardware, and batching. Figures 1 and 2 visualize this landscape together with the other evaluation scenarios.

Model	Accuracy	F1	Precision	Recall	FPR	Infer (ms)
LR	0.9911	0.9951	0.9938	0.9965	0.0611	0.0001
RF	0.9995	0.9997	0.9997	0.9998	0.0029	0.0014
XGB	0.9995	0.9997	0.9996	0.9998	0.0037	0.0003
MLP	0.9981	0.9989	0.9985	0.9994	0.0148	0.0006

Table 1: Experiment A results. Near-perfect performance across all models under conventional random splitting.

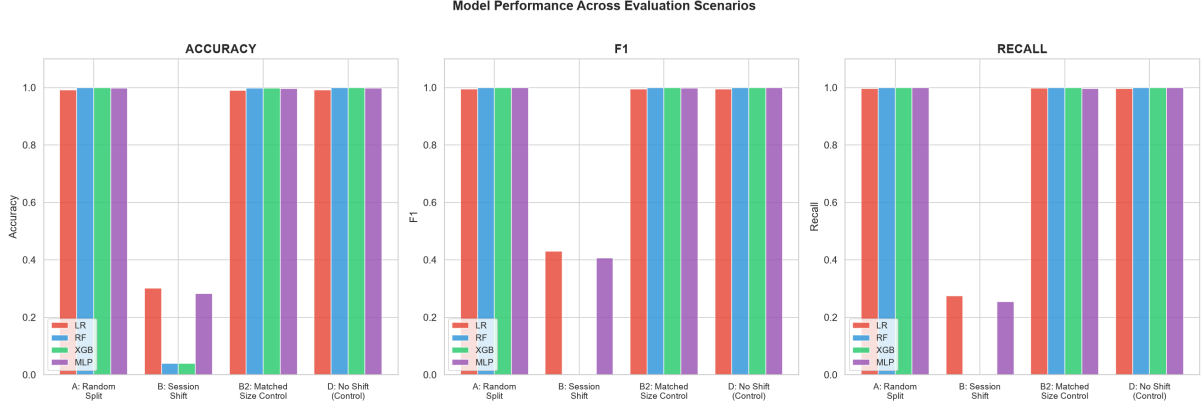


Figure 1: Model performance (accuracy, F1, recall) across the random-split, session-shift, matched-size, and no-shift evaluation scenarios.

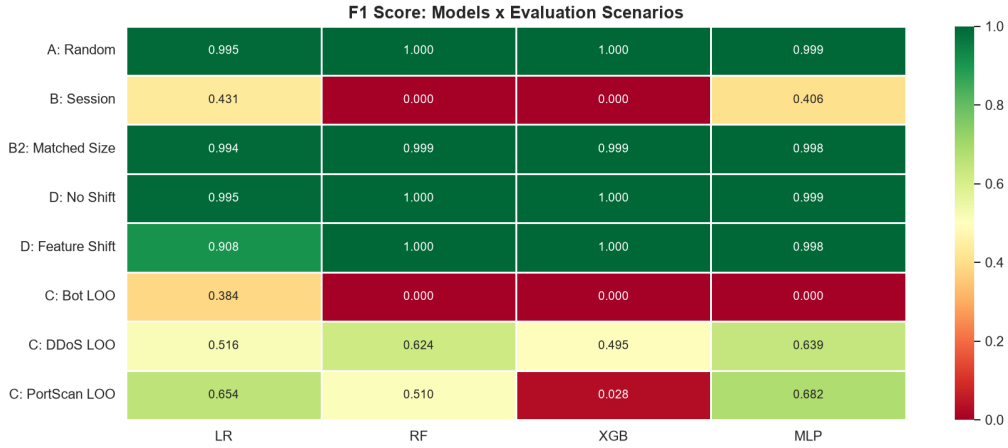


Figure 2: F1-score landscape across models and evaluation scenarios, including the attack-family leave-one-out results.

7.2 Joint Session and Attack-Family Shift (Experiment B)

Under this distribution shift, performance degrades dramatically (Table 2). Random Forest collapses to $F1 = 0.0000$ — the model predicts all traffic as benign. XGBoost similarly collapses to $F1 = 0.0003$. Logistic Regression retains partial detection ($F1 = 0.4307$) and MLP limited detection ($F1 = 0.4059$), but both miss approximately 73% of attacks. Because B compounds unseen attack classes (DDoS, PortScan) with session-level covariate shift, Experiment B4 and the Wasserstein quantification in Section 7.7 attribute this degradation to its constituent causes.

Model	Accuracy	F1	Precision	Recall	FPR	Infer (ms)
LR	0.3021	0.4307	0.9949	0.2748	0.0343	0.0001
RF	0.0393	0.0000	0.0000	0.0000	0.0040	0.0016
XGB	0.0395	0.0003	0.6282	0.0002	0.0025	0.0003
MLP	0.2833	0.4059	0.9960	0.2549	0.0249	0.0007

Table 2: Experiment B results. Severe degradation, especially for tree-based models.

The low accuracy for RF (0.0393) and XGB (0.0395) may appear paradoxical given that the test set is 96.1% attack. It is explained by the models predicting the minority class (benign) for almost all samples: a model predicting all-benign on a test set with 3.9% benign achieves ~3.9% accuracy. The low FPR values (0.0040, 0.0025) are not positive outcomes — they are symptoms of total detection failure, as the models never predict attack.

7.3 Isolating Distribution Shift from Train Size (B vs B2)

The most important finding comes from comparing B with its matched-size control B2 (Table 3). Both use 13,736 training samples; the only difference is whether training data comes from the same distribution as the test set (B2) or a different one (B).

Model	B: Session Shift	B2: Same Distribution
LR	0.4307	0.9942
RF	0.0000	0.9992
XGB	0.0003	0.9991
MLP	0.4059	0.9979

Table 3: Distribution shift isolation. B and B2 use the same training set size (13,736) with zero train/test overlap.

The gap between B and B2 is striking: with the same 13,736 training samples, RF achieves F1 = 0.9992 under same-distribution conditions (B2) but F1 = 0.0000 under distribution shift (B). This provides strong evidence that the degradation is attributable to distributional differences rather than insufficient training data.

The implication is clear: 13,736 training samples are not insufficient — they suffice for near-perfect performance when they match the test distribution. The problem is that the training data, though from the same dataset, comes from sessions and attack types that do not represent the test distribution. This is a fundamentally different problem than data scarcity and requires different solutions (distribution-aware evaluation, continual retraining, or domain adaptation) rather than simply collecting more labeled data. Figure 3 shows this contrast directly.

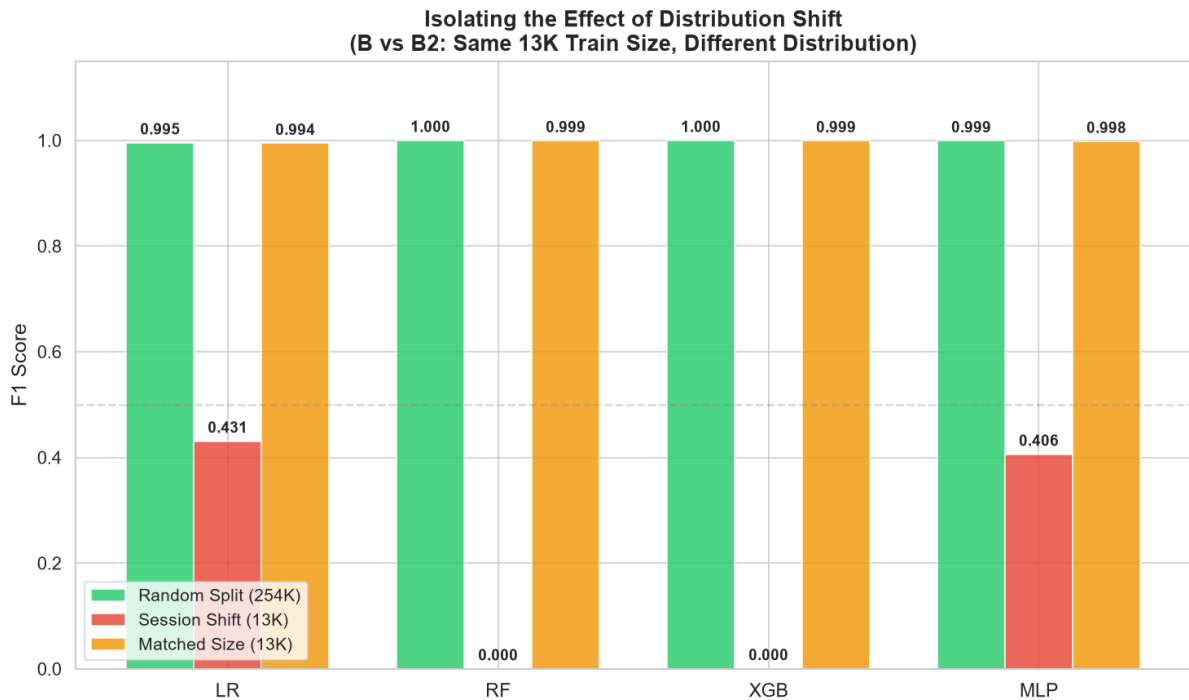


Figure 3: Isolating distribution shift from train size. B and B2 use identically sized training sets (13,736 samples) whose distributions differ.

7.4 Separating Unseen-Class Shift from Covariate Shift (Experiment B4)

Experiment B's test set contains attack families absent from training, so its collapse could in principle be attributed to either novel attack signatures or displaced feature values. Experiment B4 removes the unseen-class component: DDoS is present in training, and only the test feature values are perturbed (Table 4).

Model	B4a: Same Dist.	B4b: Covariate Shift	Delta
LR	0.9990	0.9982	-0.0007
RF	0.9999	0.9997	-0.0002
XGB	0.9999	0.9997	-0.0003
MLP	0.9998	0.9995	-0.0004

Table 4: Single-class covariate-shift control. DDoS is present in both training and test; only test features are perturbed. F1 drops are negligible compared with Experiment B.

When the attack class is present in training, feature displacement of the same design as Experiment D causes negligible degradation (F1 drops of 0.0002–0.0007 across all four models), in stark contrast to Experiment B, where the same models lost 0.57–0.99 F1. The dominant driver of B's collapse is therefore the presence of entirely novel attack families (unseen-class generalization), not feature displacement per se. The covariate-shift component of B cannot be fully separated from its class-composition change here, but B4 shows that displacement alone — even a moderate one (normalized Wasserstein 0.152, Section 7.7) — does not produce B's catastrophic failures. This frames B as an open-set recognition problem: the models have no decision boundary for attack families they have never seen.

7.5 Attack-Family Leave-One-Out (Experiment C)

When specific attack types are excluded from training (Table 5), the results reveal which attack families are most distinct from the training distribution.

Held-out	LR	RF	XGB	MLP
Bot	0.384	0.000	0.000	0.000
DDoS	0.516	0.624	0.495	0.639
PortScan	0.654	0.510	0.028	0.682

Table 5: F1 scores under attack-family leave-one-out. Bot detection is most severely affected.

The Bot leave-one-out is the most severe: all models except LR achieve F1 = 0.000 when Bot attacks are absent from training, despite 314,325 training samples from the other three classes. Bot traffic is evidently highly distinct from DDoS and PortScan in feature space, and learned patterns do not transfer. DDoS and PortScan leave-one-out degrade more gracefully, with MLP best (0.639 and 0.682), because Bot traffic in training shares more feature overlap with these families. "Unseen attack type" is thus not a uniform category: some unseen attacks are more detectable depending on their feature similarity to known attacks. Figure 4 plots the leave-one-out F1 scores; Figure 5 converts the same results into relative degradation from the baseline.

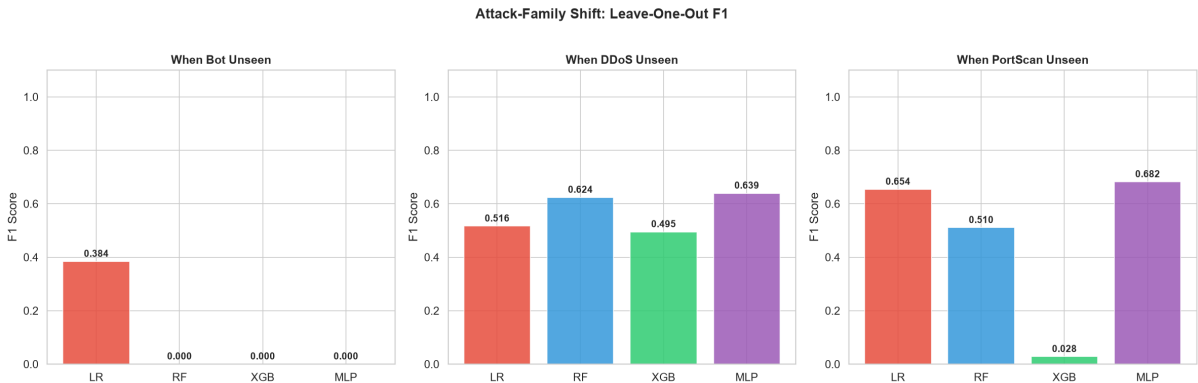


Figure 4: Attack-family leave-one-out F1 per held-out class. Bot detection is the most severely affected.

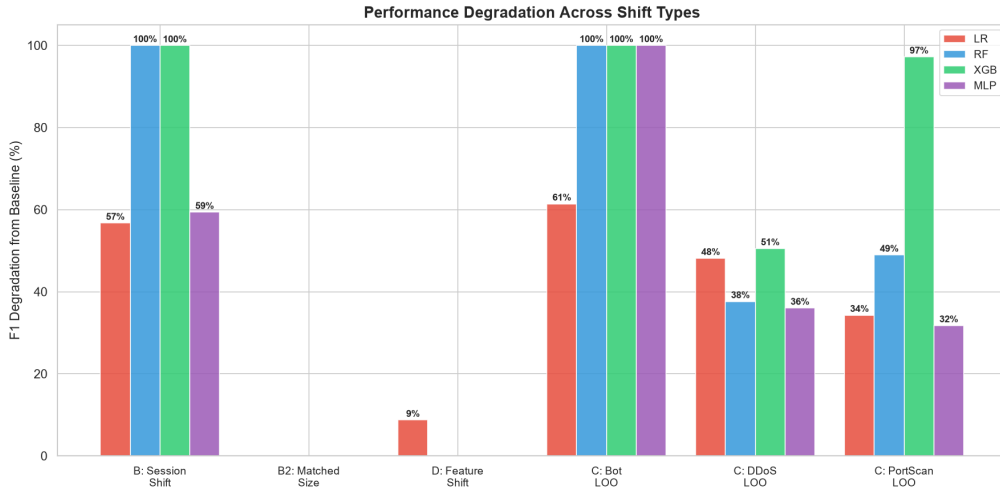


Figure 5: F1 degradation relative to the Experiment A baseline for each shift condition and model.

7.6 Synthetic Feature Perturbation (Experiment D)

Under synthetic feature perturbation (Table 6), tree-based models are largely unaffected (RF F1 drops from 0.9997 to 0.9997, XGB from 0.9997 to 0.9997), while Logistic Regression shows notable degradation (0.9951 to 0.9084).

Model	No Shift	Feature Shift
LR	0.9951	0.9084
RF	0.9997	0.9997
XGB	0.9997	0.9997
MLP	0.9989	0.9983

Table 6: Experiment D results. Tree models showed little degradation under the specific feature perturbations used in this experiment.

RF and XGBoost showed almost no degradation under the specific feature perturbations used in this experiment (F1 effectively unchanged at 0.9997). This should be interpreted as an empirical result for this particular perturbation design rather than evidence of general invariance to feature distribution changes. The differing sensitivity has a mechanical explanation: tree models split on feature ranks and thresholds, so uniformly scaling a feature — or scaling it across the board by positive factors — does not change the relative ordering of samples at a given split, leaving the decision boundaries intact; linear models, by contrast, combine features additively using learned weights, so multiplying feature magnitudes directly shifts the linear scores and their calibration to the 0.5 threshold. Logistic Regression therefore shows the expected degradation (0.9951 to 0.9084).

Notably, this contrast is informative: tree models were highly vulnerable to session/attack-family shift (Experiment B) but stable under this particular perturbation. More realistic perturbations — changed feature correlations, new features from protocol changes, or dropped features from sensor failures — could affect tree models differently.

7.7 Shift Magnitude Quantification (Wasserstein Distances)

The preceding experiments describe shifts qualitatively; Table 7 quantifies them. For each shift we report the mean normalized one-dimensional Wasserstein distance between training and test features (a random subsample of 25,000 points per side, per-feature distance divided by the pooled standard deviation).

Shift	Mean W1 (norm.)	Top shifted features	Max F1 drop
B (joint session + unseen class)	0.293	PSH Flag Count (1.01), Min Packet Length (0.81), Bwd Packet Length Min (0.77)	0.9997 (RF)
B4 (seen-class covariate shift)	0.152	Flow Duration, packet counts (as in D)	0.0008 (LR)
D (synthetic feature perturbation)	0.106	Packet Length Variance (0.59), Flow Duration (0.47), Flow IAT Std (0.47)	0.0867 (LR)

Table 7: Mean normalized Wasserstein distance between training-feature and test-feature distributions, per shift, with the largest per-feature distances and the largest observed F1 drop across the four models (baseline = the experiment's own no-shift condition: random split A for B, B4a for B4, no-shift control for D).

Three observations follow. First, B is the largest displacement (0.293) and produces the largest F1 collapse — in the expected direction. Second, the displacement-degradation relationship is strongly non-linear: B4 displaces features roughly half as much as B (0.152 vs. 0.293) yet produces a thousand-fold smaller F1 drop. Third, B's F1 drop of 0.9997 is far out of proportion to what feature displacement alone would predict from B4, confirming that unseen-class generalization, not covariate magnitude, is the primary driver of B's collapse. Figure 6 reports this relationship directly.

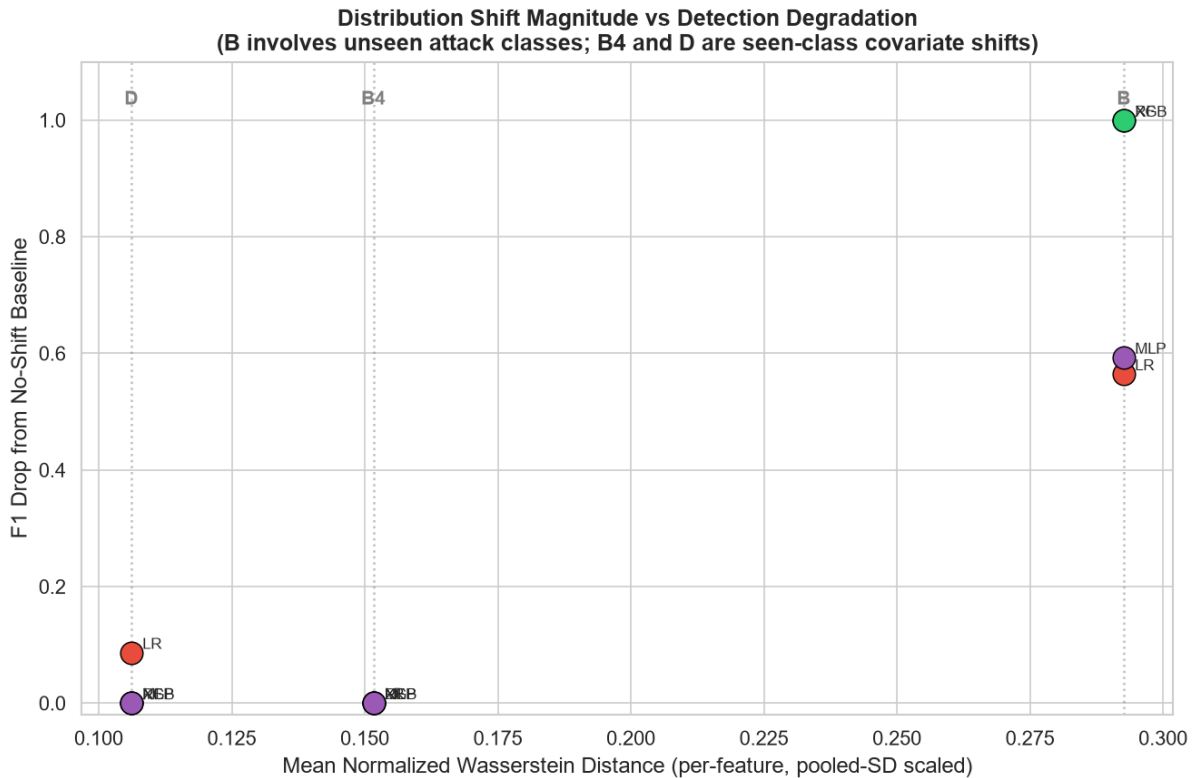


Figure 6: Mean normalized Wasserstein shift magnitude versus maximum F1 drop across the B, B4, and D shifts.

7.8 Threshold Sweep on the Shifted Test Set

The ROC-AUC analysis below shows that rankings survive even when F1 collapses at the default threshold; the sweep (Section 6.5) tests whether this signal is exploitable. Table 8 reports F1 and FPR at the default threshold (0.50), at a moderately aggressive threshold (0.05), and at an aggressive threshold (0.01); Figure 7 shows the full curves.

Model	F1 @ 0.50	FPR @ 0.50	F1 @ 0.05	FPR @ 0.05	F1 @ 0.01	FPR @ 0.01
LR	0.4307	0.0343	0.7104	0.0581	0.7115	0.1602
RF	0.0000	0.0040	0.7114	0.0399	0.8262	0.0654
XGB	0.0003	0.0025	0.4635	0.0081	0.6037	0.0219
MLP	0.4059	0.0249	0.4923	0.0368	0.6614	0.0492

Table 8: Threshold sweep on the Experiment B test set. Lowering the decision threshold recovers a substantial fraction of F1 at the cost of higher FPR; even at the best non-degenerate thresholds, all models still underperform their same-distribution baselines.

Threshold-based recalibration recovers part of the collapsed F1: Random Forest rises from 0.0000 (FPR 0.4%) at the default threshold to 0.7114 at 0.05 and 0.8262 at 0.01, with FPR climbing to 4.0% and 6.5%. The ranking signal identified by ROC-AUC is therefore real and exploitable, but recovery is neither free nor complete. First, the required thresholds (0.01–0.05) are far below the default, so near-line-rate deployments would flag a large share of traffic; the FPR cost is diluted on the 96.1%-attack test set but would be far larger in benign-dominated production traffic. Second, the naive best-F1 point on this test set is the degenerate solution of predicting every sample as attack (F1 = 0.98 at FPR = 1.0), which provides no useful detection. Third, even the best recoverable operating point (RF, 0.8262) falls below the same-distribution capability (B2, 0.9992); recalibration is a partial remedy, not a restoration of the original decision function.

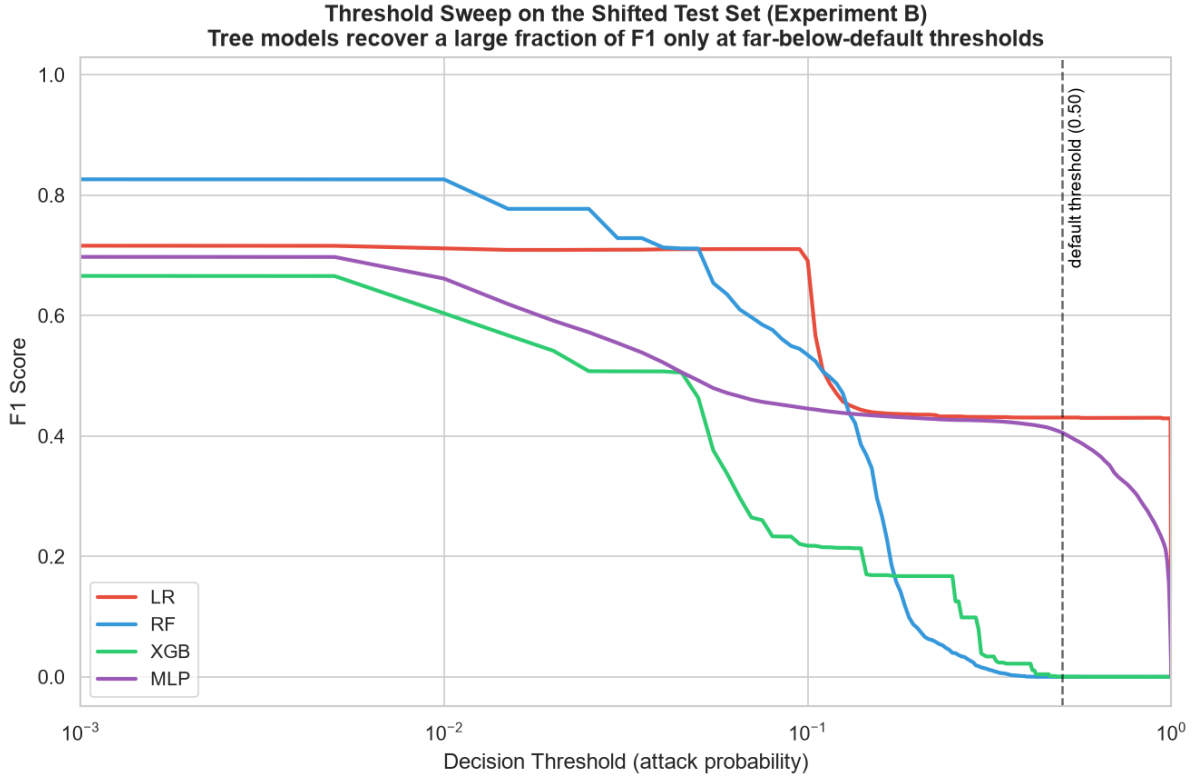


Figure 7: F1 versus decision threshold on the Experiment B test set. Tree models require far-below-default thresholds to recover useful F1.

7.9 ROC-AUC Under Shift

An important secondary finding emerges from comparing F1 and ROC-AUC under session-based shift (Table 9). Figure 8 compares ROC-AUC across scenarios.

Model	F1 (B)	ROC-AUC (B)
LR	0.4307	0.6081
RF	0.0000	0.8185
XGB	0.0003	0.8452
MLP	0.4059	0.6485

Table 9: F1 collapses while ROC-AUC retains partial signal under distribution shift.

RF achieves ROC-AUC = 0.8185 despite F1 = 0.0000, and XGB achieves 0.8452 despite F1 = 0.0003. This suggests that the models' probability rankings retain some discriminative information, while the default classification threshold may be poorly suited to the shifted test distribution. The models are not devoid of information: their operational behavior is broken, but their representations partially survive the shift. The threshold sweep in Section 7.8 confirms this interpretation: the retained ranking signal can be converted back into classification performance, but only at far-below-default thresholds with a measurable false-alarm cost, so the recoverable signal is partial. Monitoring ROC-AUC alongside F1 in production could still provide an early warning signal — distinguishing threshold miscalibration from fundamental model failure — but the sweep shows that the practical remedy is bounded.

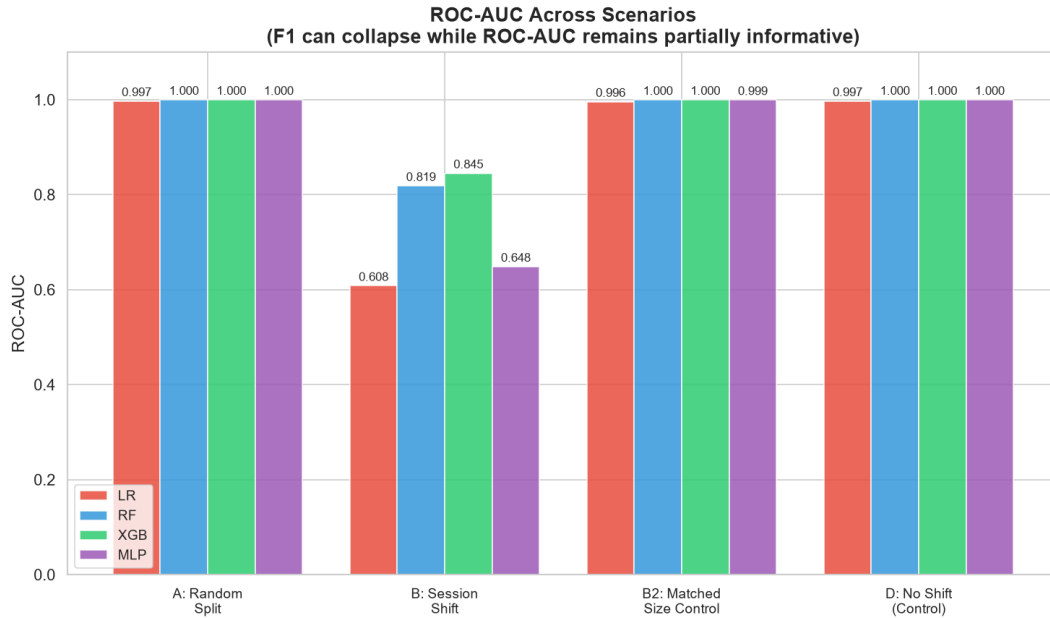


Figure 8: ROC-AUC across evaluation scenarios. F1 can collapse while ROC-AUC retains a partial ranking signal.

8. Security Implications

8.1 The False Sense of Security Problem

The central security implication is that conventional evaluation can produce a false sense of security: a model evaluated as 99.97% effective may provide zero detection capability in deployment, a predictable consequence of the gap between evaluation and deployment conditions.

The paradox is that the models with the highest benchmark scores in this experiment (RF and XGB at 0.9997) also failed most catastrophically under the evaluated distribution shift: predicting all traffic as benign means they provide no security value while appearing, by conventional metrics, to be the strongest performers. This supports the emphasis of the NIST AI Risk Management Framework and NIST IR 8596 on continuous monitoring and adaptation: a model that performs well at one point may fail when deployed in a changed environment, and standard evaluation does not detect this before deployment.

8.2 Failure Mode Analysis

The failure modes differ by architecture, with different security implications:

Tree-based models (RF, XGB): Fail by predicting all traffic as benign (recall approaches 0), the worst possible failure mode: every attack passes undetected while the model provides the illusion of a clean network.

Linear and neural models (LR, MLP): Fail by detecting only a subset of attacks (recall approximately 0.25–0.27) with high precision. The model misses approximately 73% of attacks but rarely generates false alarms. While imperfect, this failure mode provides partial visibility into threats.

From an operational standpoint, the tree-based failure mode is strictly worse: a model that never raises an alarm provides no security value, and deploying it may increase risk by replacing signature-based systems that would have caught some attacks. The practical implication for model selection in cloud security is that **the model that performs best under conventional evaluation may not be the most robust choice for deployment**. Organizations should evaluate candidate models under distribution-aware conditions before deployment, not solely on benchmark performance. If only conventional evaluation is feasible, the results suggest that LR or MLP showed less catastrophic degradation than tree-based models under the evaluated shift, despite their lower benchmark scores.

8.3 Implications for Cloud Deployment

Deploying ML-based intrusion detection in cloud environments introduces additional complexities. In centralized architectures where traffic from multiple services is aggregated into a single model, the distribution shift problem is amplified. Our finding that tree-based models collapse under even a single session-based shift suggests that models requiring broad generalization across diverse traffic types may be particularly vulnerable.

Inference latency. Inference times differ by an order of magnitude (LR 0.0001 ms/sample, MLP 0.0006, RF 0.0014). At cloud scale, where millions of flow records may be classified per minute, these differences are operationally significant — but the latency advantage of LR is meaningless if the model fails to detect attacks under shift, so the speed-robustness trade-off should be informed by distribution-aware evaluation.

Threshold sensitivity. The threshold sweep confirms that the retained ranking signal is exploitable but bounded. Because all four models retained ROC-AUC well above 0.5 under shift, an operator could recover a large fraction of F1 by lowering the threshold — but only by moving far from 0.50 (to roughly 0.01–0.05), at which point the FPR and the fraction of traffic flagged rise materially: Random Forest's FPR increases from 0.4% at threshold 0.50 to 6.5% at threshold 0.01. A real deployment operating in benign-dominated traffic would see a proportionally larger false-alarm volume than the 96.1%-attack test set suggests; selecting the operational threshold without ground-truth labels from the target distribution remains a practical challenge.

Hybrid architectures. The differing failure modes across architectures suggest that ensemble approaches combining multiple model types may provide more robust detection. A system that combines tree-based models (fast inference, high benchmark performance) with linear models (graceful degradation under shift) could potentially leverage the strengths of each while mitigating their individual weaknesses under distribution shift.

9. Limitations

Dataset scope. We use only the Friday subset of CICIDS2017 (BENIGN, Bot, DDoS, PortScan). The full dataset includes additional attack categories (Brute Force, Heartbleed, Web Attacks, Infiltration) spread across Monday through Friday, which may produce different shift patterns, and our findings may not generalize to datasets with different attack types or network configurations. Future work should replicate this analysis on UNSW-NB15, CICIDS2018, and real-world production traffic.

Session-shift construction. The session-based split in Experiment B partitions by attack type (Bot = morning, DDoS/PortScan = afternoon). This approximates but does not perfectly replicate a pure

timestamp-level temporal split, since preprocessing concatenates rows by class, partially destroying original temporal ordering. We therefore label it *session and attack-family shift* rather than *temporal shift*.

Train-test size imbalance. The session split produces substantially different training and test set sizes (13,736 vs. 298,609). While the B2 control confirms that distribution shift, not train size, is the primary cause of degradation, the imbalance also shifts class proportions (training is 85.7% benign; test is 3.9% benign), an additional variable in the experimental design.

Model configurations. We evaluate four architectures with fixed, untuned hyperparameters; different configurations might exhibit different robustness profiles. However, the complete collapse of RF and XGB under session shift suggests the main finding is robust to moderate hyperparameter variation. We do not evaluate recurrent, attention-based, or graph-neural architectures, which may behave differently, though the ML focus here is deliberate (Section 5.3).

Feature shift design. The synthetic perturbation in Experiment D is limited: it multiplies flow-level magnitudes by positive factors while preserving feature correlations. Tree models showed little degradation under this transformation, but this does not establish robustness to joint-distribution, correlation, feature-availability, or sensor changes, any of which could reorder samples across tree splits. Joint-distribution shifts remain untested.

Binary classification focus. While we report multi-class results, the primary analysis uses binary classification (BENIGN vs. ATTACK). Distribution shift may affect multi-class performance differently, especially class-specific detection.

10. Conclusion

The results in this paper show that conventional random-split evaluation can substantially overestimate the operational effectiveness of ML-based intrusion detection models. Models that score 0.999+ F1 on a random split collapsed to 0.000 under distribution shift, while a matched-size control trained on the same number of samples from the same distribution held at 0.999. The gap is distributional, not a shortage of training data.

The failure modes differ by architecture in ways that have direct security implications. Tree-based models, which achieve the highest benchmark scores, exhibit the most catastrophic failure (complete detection collapse), while linear and neural models retain partial detection capability. A single-class covariate-shift control shows that feature displacement on a seen class is comparatively benign, attributing the collapse in the joint session/attack-family scenario primarily to unseen attack classes rather than covariate magnitude. The secondary finding that ROC-AUC remains partially informative while F1 collapses suggests that the models retain discriminatory information despite broken operational behavior; a threshold sweep confirms this signal can be partially recovered, but only at far-below-default thresholds whose false-alarm cost must be borne in benign-dominated deployment traffic.

These findings have implications for both research methodology and operational deployment. For researchers, the results support distribution-aware evaluation as a complement to conventional benchmarking: random-split evaluation remains a useful upper bound on capability, but it should not be the sole methodology. Any study reporting high performance on an IDS benchmark should supplement random-split results with at least one distribution-shift evaluation. For practitioners, the results caution against relying on benchmark performance alone when choosing deployment models: organizations should evaluate candidates under conditions matching their deployment environment, monitor for distribution drift in production, maintain fallback detection mechanisms independent of ML assumptions, and establish retraining protocols that account for the temporal evolution of traffic and attack patterns.

Several directions for future work emerge. First, the framework should be replicated on additional datasets (UNSW-NB15, CICIDS2018) and architectures (recurrent networks, transformers, and calibrated variants such as Platt-scaled or isotonic-regression classifiers). Second, distribution shift detection methods that alert operators when operating conditions diverge from training conditions would complement robustness evaluation, with the ROC-AUC-versus-F1 gap identified here as a candidate early-warning signal. Third, domain adaptation and continual learning approaches that address shift during deployment could provide

more robust alternatives to static deployment. Fourth, adaptive threshold strategies informed by the sweep results — including operating-point selection that accounts for false-alarm cost in benign-dominated traffic — could operationalize the ROC-AUC finding.

The broader lesson is that evaluation methodology itself shapes the conclusions we draw about model capability: a model evaluated under conditions that do not match deployment is effectively a different model. As ML-based intrusion detection matures, standardized robustness evaluation protocols — analogous to adversarial robustness benchmarks in computer vision — become increasingly important for closing the gap between benchmark performance and operational security. The framework presented here, combining distribution-shift evaluation with matched-size and covariate-shift controls, Wasserstein quantification, and failure mode analysis, provides a template for such protocols.

Declarations

Author contributions: Dhansika.R is the sole author: conceptualization, research direction, dataset selection, experimental design decisions, results review and interpretation, and writing.

Use of Generative AI and AI-Assisted Technologies: Generative AI tools were used as guided assistance throughout this project. The author conceived the research direction, selected the dataset, and made all high-level experimental design decisions in collaboration with the AI assistant tool *opencode*, including the matched-size control (B2) that isolates distribution shift from training set size and the covariate-shift control (B4) that separates unseen attack classes from feature displacement. The author directed the experiments, reviewed every result, and verified the reported metrics against the experiment logs. AI assistance was used to draft the Python experiment scripts, generate the figures, and edit and polish the manuscript prose; all findings were reviewed and validated by the author, who takes full responsibility for the scientific validity and integrity of this work.

Data and code availability: All code (preprocessing, experiments, verification, and figure generation) and the publicly available CICIDS2017 dataset (Sharafaldin et al., 2018) are available from the author on request. The preprocessing pipeline is fully scripted and reproducible.

Competing interests: The author declares no competing interests.

Funding: This research received no specific grant from any funding agency.

References

- Ahmad, Z., Shahid Khan, A., Wai Shiang, C., Abdullah, J., & Ahmad, F. (2021). Network intrusion detection system: A systematic study of machine learning and deep learning approaches. *Transactions on Emerging Telecommunications Technologies*, 32(1), e4150.
- Akinola, O., Gber, T. E., Oluwadare, T., & Yoshida, M. (2022). Adaptive intrusion detection systems: A comprehensive review of approaches, challenges, and evaluation methodologies. *Sensors*, 22(15), 5775.
- Coronges, K., et al. (2024). A comprehensive survey of adversarial machine learning in network intrusion detection systems. *arXiv preprint*.
- Geirhos, R., et al. (2020). Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2(11), 665–673.
- Nibouchia, O., et al. (2024). Temporal evaluation in intrusion detection: A systematic literature review. *Computers & Security*, 137, 103551.
- NIST. (2023). AI Risk Management Framework (AI RMF 1.0). National Institute of Standards and Technology.
- NIST. (2025). NISTIR 8596: Cybersecurity Framework Profile for Artificial Intelligence. National Institute of Standards and Technology.

- Sadhnani, M., et al. (2024). Explainable AI for concept drift detection in network intrusion detection systems. *Computers & Security*, 137, 103568.
- Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A. (2018). Toward generating a new intrusion detection dataset and intrusion traffic characterization. *Proceedings of ICISSP*, 108–116.
- Shyaa, S. H., et al. (2024). Concept drift in intrusion detection: A comprehensive survey. *IEEE Access*, 12, 45000–45025.
- Tavallaee, M., Bagheri, E., Lu, W., & Ghorbani, A. A. (2009). A detailed analysis of the KDD CUP 99 data set. *Proceedings of IEEE Symposium on Computational Intelligence for Security and Defense Applications*, 1–6.
- Xu, C., et al. (2024). adversarial machine learning in intrusion detection systems: A comprehensive survey. *ACM Computing Surveys*, 56(2), 1–36.